

Binary search

Luca Didaci

Table of contents

1	Binary search	1
1.1	Binary search	1
1.2	Solution (iterative)	1
1.3	Solution (recursive)	2
1.4	Solution (python)	3
1.5	Ricorrenza fondamentale	3

1 Binary search

1.1 Binary search

Binary search is an efficient algorithm used to find a specific element in a **sorted** array. It works by repeatedly dividing the search interval in half until the desired element is found or the interval is empty.

- Start with the entire array.
- Compare the middle element of the array with the target value.
- If `middle_element == target`, return its index.
- If `target < middle_element`, continue the search on the left half of the array.
- If `target > middle_element`, continue the search on the right half of the array.

Repeat until the target is found or the search interval is empty.

1.2 Solution (iterative)

```
#include <stdio.h>

// binary search on a static array
int binarySearch(int arr[], int size, int target) {
    int left = 0;
    int right = size - 1;

    while (left <= right) {
        int idx_middle = left + (right - left) / 2;

        // Check if target is present at idx_middle
        if (arr[idx_middle] == target)
            return idx_middle;

        // If target is greater, ignore left half
        if (arr[idx_middle] < target) {
            left = idx_middle + 1;
            // right does not change
        }
    }
}
```

1 Binary search

```
        else { // If target is smaller, ignore right half
            // left does not change
            right = idx_middle - 1;
        }
    }

    // If target is not present in array
    return -1;
}

int main() {
    int arr[] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 12;
    int index = binarySearch(arr, size, target);
    if (index != -1)
        printf("Element %d found at index %d.\n", target, index);
    else
        printf("Element %d not found in the array.\n", target);
    return 0;
}
```

1.3 Solution (recursive)

```
#include <stdio.h>

// Recursive function to perform binary search on a static array
int binarySearchRecursive(int arr[], int left, int right, int target) {
    if (right >= left) {
        int idx_middle = left + (right - left) / 2;

        // If the target is present at the middle
        if (arr[idx_middle] == target)
            return idx_middle;

        // If the target is smaller than arr[idx_middle], then it can only be present in the left subarray
        if (arr[idx_middle] > target)
            return binarySearchRecursive(arr, left, idx_middle - 1, target);

        // Else the target can only be present in the right subarray
        return binarySearchRecursive(arr, idx_middle + 1, right, target);
    }
}
```

```

    // If the element is not present in the array
    return -1;
}

int main() {
    int arr[] = {2, 4, 6, 8, 10, 12, 14, 16, 18, 20};
    int size = sizeof(arr) / sizeof(arr[0]);
    int target = 12;
    int index = binarySearchRecursive(arr, 0, size - 1, target);
    if (index != -1)
        printf("Element %d found at index %d.\n", target, index);
    else
        printf("Element %d not found in the array.\n", target);
    return 0;
}

```

1.4 Solution (python)

```

def ricerca_binaria(lista, target):
    sx = 0
    dx = len(lista) - 1
    while sx <= dx:
        i_medio = int ((sx+dx)/2)
        v = lista[i_medio]
        if v == target:
            return i_medio
        else:
            if target > v:
                sx = i_medio + 1
            else:
                dx = i_medio - 1
    return None

lista = [10, 20, 30, 40, 50]
ricerca_binaria(lista, 11)

```

1.5 Ricorrenza fondamentale

Algoritmo che dimezza input ad ogni passo

1 Binary search

$$C_N = C_{N/2} + 1$$

$$N \geq 2; N = 2^n$$

$$C_N = C_{2^n} =$$

$$= C_{2^{n-1}} + 1 =$$

$$= C_{2^{n-2}} + 2 =$$

$$= C_{2^{n-3}} + 3 =$$

$$= \dots$$

$$= C_{2^0} + n =$$

$$= 1 + n = 1 + \log_2 N$$
